Virtualization and Cloud Computing
Assignment 3: Local VM to Cloud Auto Scaling

Name: Om Kumar (B22CS081)
Cloud Platform Used: Google Cloud Platform (GCP)

1 Introduction

Modern cloud systems are designed to dynamically adapt to varying workload demands, and one of the most critical mechanisms enabling this adaptability is auto-scaling. Auto-scaling allows computational resources to be provisioned or released automatically based on system load, ensuring both performance efficiency and cost optimization. In this assignment, I implemented a hybrid cloud system in which a local virtual machine continuously monitors its own resource utilization and triggers provisioning of additional resources in a public cloud when predefined thresholds are exceeded. This paradigm is commonly referred to as cloud bursting, where local infrastructure is extended into the cloud during peak demand conditions.

The implementation goes beyond a simple threshold-triggered scaling mechanism and incorporates controlled load generation, bidirectional scaling (both scale-up and scale-down), and a stabilization strategy using cooldown periods. This makes the system closer to real-world production scenarios, where uncontrolled scaling can lead to instability and increased operational costs.

2 Local Virtual Machine Setup

The local environment was created using VMware Workstation Pro with Kali Linux as the operating system. The virtual machine was configured with two virtual CPUs and two gigabytes of RAM (initially set as 4GB, but due to my PC overload, I reduced it). This configuration was intentionally chosen to allow observable scaling behavior within a limited resource environment. By restricting available computational capacity, it becomes easier to reach utilization thresholds and study system response under controlled load conditions.

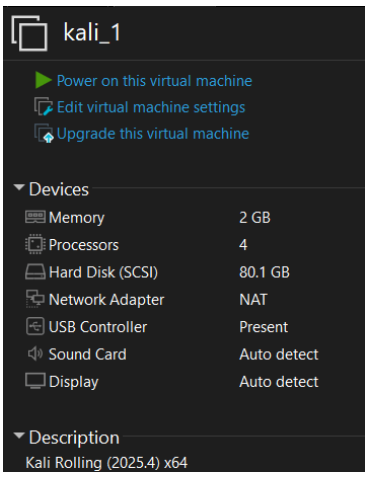


Figure 1: Local Kali Linux Virtual Machine Setup in VMware Workstation Pro

The local virtual machine serves as the primary execution environment for monitoring, decision-making, and load generation. Unlike cloud instances, which are dynamically provi-

sioned, the local VM represents a fixed resource pool whose limitations motivate the need for external scaling.

During the setup, I initially attempted to install the Google Cloud CLI using standard package installation commands such as `sudo apt install google-cloud-cli`. However, this approach failed because Kali Linux does not include the required repositories for the Google Cloud SDK by default.

To resolve this issue, I followed the official installation method by manually adding the Google Cloud SDK repository and its signing key. The following commands were used:

```
sudo apt install -y apt-transport-https ca-certificates gnupg curl

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg

echo "deb[signed-by=/usr/share/keyrings/cloud.google.gpg]\
http://packages.cloud.google.com/apt_cloud-sdk_main" | \
sudo tee /etc/apt/sources.list.d/google-cloud-sdk.list

sudo apt update
sudo apt install -y google-cloud-cli
```

After installation, I verified the setup and authenticated the CLI using:

```
gcloud version
gcloud auth login
gcloud config set project vcc-assignment-3
gcloud compute instance-groups list
```

This allowed the local VM to interact with Google Cloud resources programmatically.

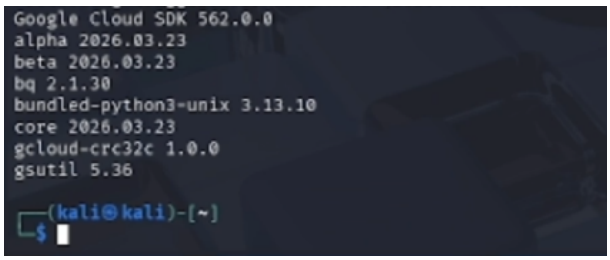


Figure 2: Successful Installation and Verification of gcloud CLI

3 Resource Monitoring Mechanism

A critical component of any auto-scaling system is the ability to accurately monitor resource usage and make decisions based on real-time data. Instead of relying on heavy monitoring frameworks such as Prometheus and Grafana, I implemented a lightweight monitoring mechanism using shell scripting. This design choice was made to reduce system complexity and overhead while retaining full control over the monitoring logic.

The monitoring script periodically collects CPU utilization using the `top` command and memory utilization using the `free` command. CPU utilization is particularly significant as a scaling signal because it directly reflects computational demand and is widely used in industry for triggering scaling decisions. Memory utilization was also included to account for scenarios where applications may be memory-bound rather than CPU-bound.

Threshold values were carefully selected based on standard practices. A scale-up threshold of 75% was chosen as it represents a balance between responsiveness and stability. Setting the threshold too low would result in premature scaling and unnecessary resource usage, whereas

setting it too high could lead to performance degradation before scaling is triggered. Similarly, a scale-down threshold of 30% was chosen to ensure that underutilized resources are released, thereby improving cost efficiency.

To prevent rapid oscillations between scaling up and scaling down, a cooldown period of 60 seconds was implemented. This stabilization mechanism ensures that once a scaling decision is made, the system waits for a fixed interval before taking further actions. Without such a mechanism, the system could exhibit unstable behavior, frequently adding and removing instances in response to minor fluctuations in load.

```
#!/bin/bash
UP_THRESHOLD=75
DOWN_THRESHOLD=30
COOLDOWN=60
LAST_TRIGGER=0
LOG_FILE="scaling.log"
ZONE="asia-south2-a"
INSTANCE_GROUP="om-a3-mig"
MAX_INSTANCES=5
MIN_INSTANCES=1

echo "Monitoring␣started..." >> $LOG_FILE

while true; do
    cpu_usage=$(top -bn1 | grep "Cpu(s)" | \
    sed "s/.*,*\([0-9.]*\)%*id.*/\1/" | \
    awk '{print_100-$1}')
    mem_usage=$(free | awk '/Mem:/{print($3/$2)*100}')
    timestamp=$(date)
    current_time=$(date +%s)
    echo "$timestamp|CPU:$cpu_usage%|MEM:$mem_usage%" >> $LOG_FILE
    INSTANCE_COUNT=$(gcloud compute instance-groups managed list-
       instances $INSTANCE_GROUP \
    --zone=$ZONE --format="value(instance)" | wc -l)

    # SCALE UP
    if (( $(echo "$cpu_usage>$UP_THRESHOLD" | bc -l) )) || \
       (( $(echo "$mem_usage>$UP_THRESHOLD" | bc -l) )); then
        if (( current_time - LAST_TRIGGER > COOLDOWN )); then
            if [ "$INSTANCE_COUNT" -lt "$MAX_INSTANCES" ]; then
                NEW_SIZE=$((INSTANCE_COUNT + 1))
                echo "$timestamp|Scaling_UP_to_$NEW_SIZE" >> $LOG_FILE
                gcloud compute instance-groups managed resize
                   $INSTANCE_GROUP \
                --size=$NEW_SIZE --zone=$ZONE
                LAST_TRIGGER=$current_time
            fi
        fi
    fi

    # SCALE DOWN
    if (( $(echo "$cpu_usage<$DOWN_THRESHOLD" | bc -l) )) && \
       (( current_time - LAST_TRIGGER > COOLDOWN )); then
        if [ "$INSTANCE_COUNT" -gt "$MIN_INSTANCES" ]; then
            NEW_SIZE=$((INSTANCE_COUNT - 1))
            echo "$timestamp|Scaling_DOWN_to_$NEW_SIZE" >> $LOG_FILE
            gcloud compute instance-groups managed resize
               $INSTANCE_GROUP \
            --size=$NEW_SIZE --zone=$ZONE
```

```
            LAST_TRIGGER=$current_time
        fi
    fi

    sleep 10
done
```

4 Cloud Infrastructure Configuration

Google Cloud Platform was selected as the public cloud provider due to its robust support for virtual machine management, instance templating, and scalable infrastructure services. A dedicated project was created, and the Compute Engine API was enabled to facilitate virtual machine provisioning.

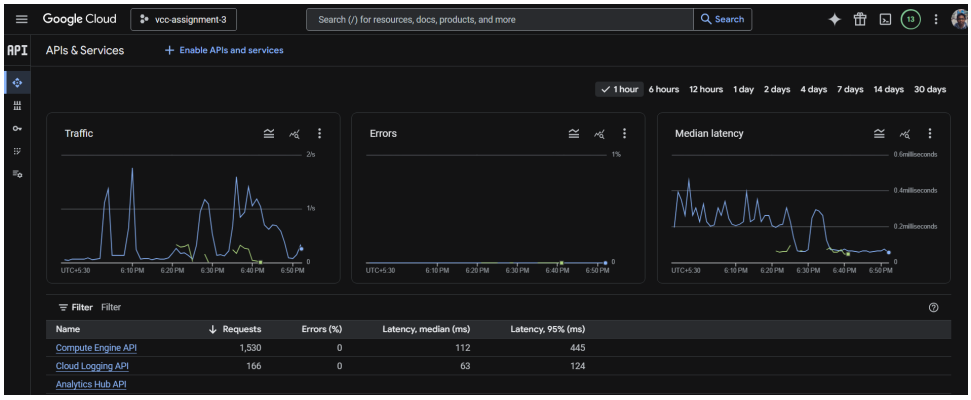


Figure 3: GCP Project Creation and API Enablement

An instance template named `om-a3-template` was created in the asia-south2 (Delhi) region. The choice of region was influenced by geographical proximity, which helps reduce network latency and improves responsiveness. The template defines a standardized configuration for all virtual machines created during scaling. It includes an e2-medium machine type, which is a cost-effective general-purpose configuration suitable for moderate workloads.

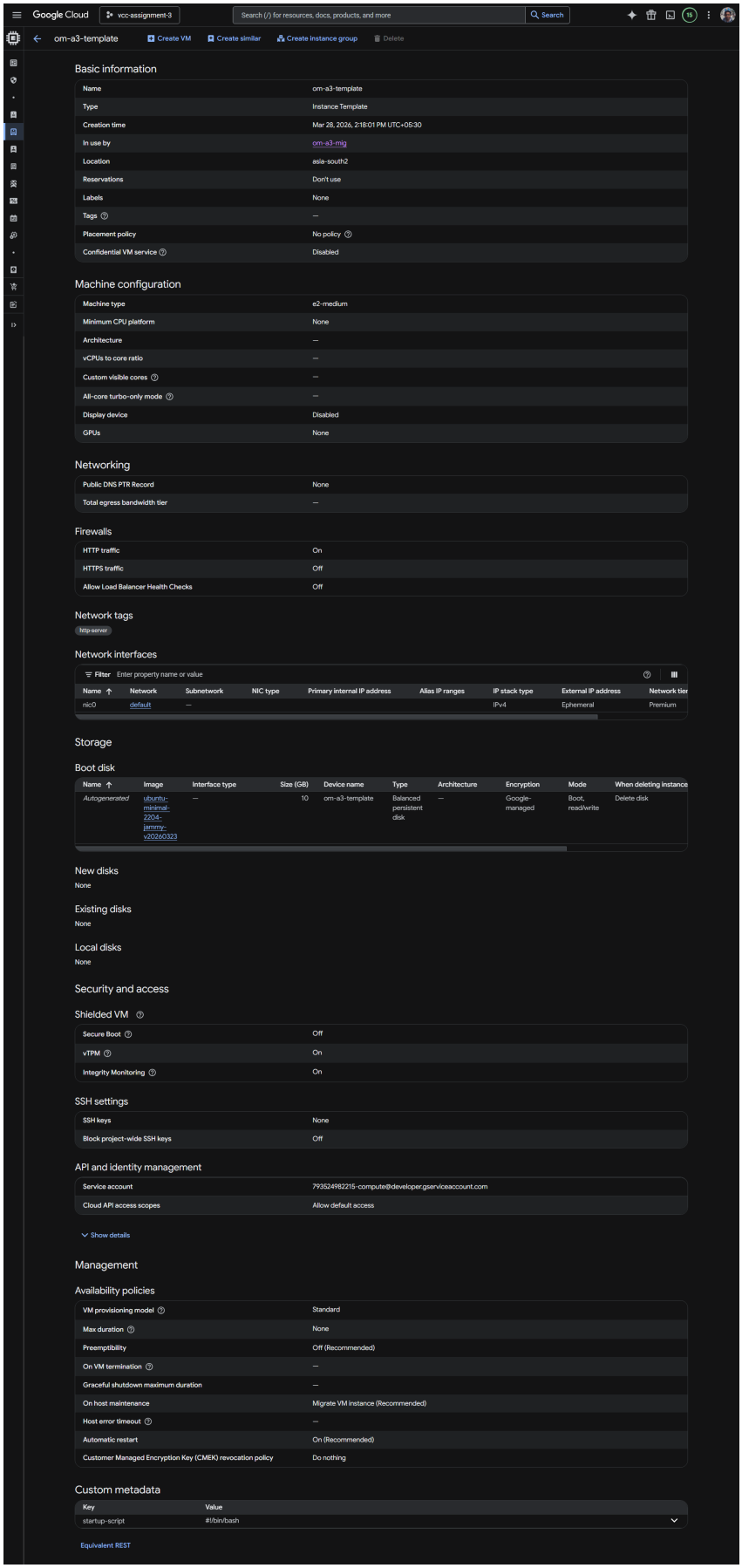


Figure 4: Instance Template Configuration (om-a3-template)

A startup script was embedded within the instance template to automate the deployment of a sample web application. This script installs the Apache web server and deploys a simple web page to verify that newly created instances are functional and serving requests correctly. The use of a startup script ensures that every instance becomes operational immediately after creation without requiring manual intervention, thereby aligning with the principles of automated infrastructure.

```
#!/bin/bash
apt update
apt install -y apache2

cat <<EOF > /var/www/html/index.html
<h1>Hybrid Auto Scaling System</h1>
<p>Instance Hostname: $(hostname)</p>
<p>Instance IP: $(hostname -I)</p>
EOF

systemctl start apache2
systemctl enable apache2
```

A stateless managed instance group was created using this template. Stateless configuration was chosen because it allows instances to be created and deleted freely without concerns about data persistence, making it ideal for scalable web-based workloads.

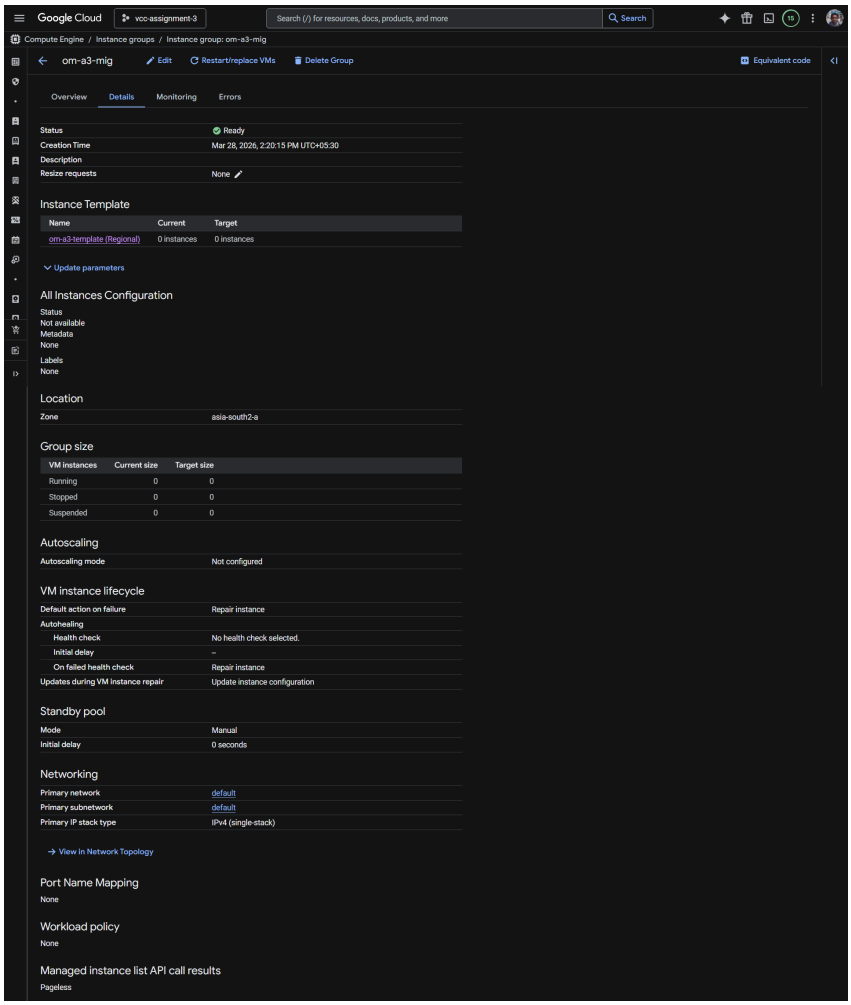


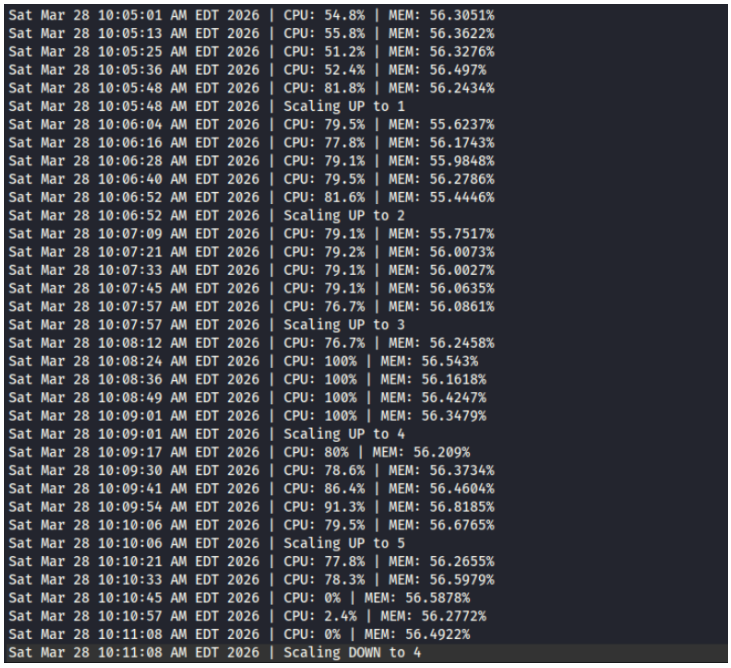
Figure 5: Managed Instance Group (om-a3-mig) Configuration

5 Auto Scaling Strategy

Auto-scaling was implemented through a custom lightweight monitoring and control script that interfaces with Google Cloud using the `gcloud` command-line tool. This approach provides fine-grained control over scaling decisions and allows integration with locally collected metrics.

When the monitored CPU or memory utilization exceeds 75%, the script triggers a scaling operation that increases the number of instances in the managed instance group. This represents a scale-up operation, where additional computational resources are provisioned to handle increased demand. Conversely, when utilization drops below 30%, the script reduces the number of instances, thereby releasing unused resources.

Unlike simplistic implementations that only support scaling up, this system supports bidirectional scaling. This is particularly important in real-world scenarios, where workloads are dynamic and resources must be adjusted both upward and downward to maintain efficiency. The inclusion of a cooldown mechanism ensures that scaling decisions are stable and not influenced by short-term fluctuations.



```
Sat Mar 28 10:05:01 AM EDT 2026 | CPU: 54.8% | MEM: 56.3051%
Sat Mar 28 10:05:13 AM EDT 2026 | CPU: 55.8% | MEM: 56.3622%
Sat Mar 28 10:05:25 AM EDT 2026 | CPU: 51.2% | MEM: 56.3276%
Sat Mar 28 10:05:36 AM EDT 2026 | CPU: 52.4% | MEM: 56.497%
Sat Mar 28 10:05:48 AM EDT 2026 | CPU: 81.8% | MEM: 56.2434%
Sat Mar 28 10:05:48 AM EDT 2026 | Scaling UP to 1
Sat Mar 28 10:06:04 AM EDT 2026 | CPU: 79.5% | MEM: 55.6237%
Sat Mar 28 10:06:16 AM EDT 2026 | CPU: 77.8% | MEM: 56.1743%
Sat Mar 28 10:06:28 AM EDT 2026 | CPU: 79.1% | MEM: 55.9848%
Sat Mar 28 10:06:40 AM EDT 2026 | CPU: 79.5% | MEM: 56.2786%
Sat Mar 28 10:06:52 AM EDT 2026 | CPU: 81.6% | MEM: 55.4446%
Sat Mar 28 10:06:52 AM EDT 2026 | Scaling UP to 2
Sat Mar 28 10:07:09 AM EDT 2026 | CPU: 79.1% | MEM: 55.7517%
Sat Mar 28 10:07:21 AM EDT 2026 | CPU: 79.2% | MEM: 56.0073%
Sat Mar 28 10:07:33 AM EDT 2026 | CPU: 79.1% | MEM: 56.0027%
Sat Mar 28 10:07:45 AM EDT 2026 | CPU: 79.1% | MEM: 56.0635%
Sat Mar 28 10:07:57 AM EDT 2026 | CPU: 76.7% | MEM: 56.0861%
Sat Mar 28 10:07:57 AM EDT 2026 | Scaling UP to 3
Sat Mar 28 10:08:12 AM EDT 2026 | CPU: 76.7% | MEM: 56.2458%
Sat Mar 28 10:08:24 AM EDT 2026 | CPU: 100% | MEM: 56.543%
Sat Mar 28 10:08:36 AM EDT 2026 | CPU: 100% | MEM: 56.1618%
Sat Mar 28 10:08:49 AM EDT 2026 | CPU: 100% | MEM: 56.4247%
Sat Mar 28 10:09:01 AM EDT 2026 | CPU: 100% | MEM: 56.3479%
Sat Mar 28 10:09:01 AM EDT 2026 | Scaling UP to 4
Sat Mar 28 10:09:17 AM EDT 2026 | CPU: 80% | MEM: 56.209%
Sat Mar 28 10:09:30 AM EDT 2026 | CPU: 78.6% | MEM: 56.3734%
Sat Mar 28 10:09:41 AM EDT 2026 | CPU: 86.4% | MEM: 56.4604%
Sat Mar 28 10:09:54 AM EDT 2026 | CPU: 91.3% | MEM: 56.8185%
Sat Mar 28 10:10:06 AM EDT 2026 | CPU: 79.5% | MEM: 56.6765%
Sat Mar 28 10:10:06 AM EDT 2026 | Scaling UP to 5
Sat Mar 28 10:10:21 AM EDT 2026 | CPU: 77.8% | MEM: 56.2655%
Sat Mar 28 10:10:33 AM EDT 2026 | CPU: 78.3% | MEM: 56.5979%
Sat Mar 28 10:10:45 AM EDT 2026 | CPU: 0% | MEM: 56.5878%
Sat Mar 28 10:10:57 AM EDT 2026 | CPU: 2.4% | MEM: 56.2772%
Sat Mar 28 10:11:08 AM EDT 2026 | CPU: 0% | MEM: 56.4922%
Sat Mar 28 10:11:08 AM EDT 2026 | Scaling DOWN to 4
```

Figure 6: Scaling Logs Showing Scale-Up and Scale-Down Events

6 Interactive Load Generation

To enhance the clarity and effectiveness of demonstration, an interactive web-based control panel was developed using Flask. Traditional approaches rely on manually executing stress commands in the terminal, which can be less intuitive and harder to demonstrate. The control panel provides a user-friendly interface through which load can be increased or decreased incrementally.

Since Kali Linux does not provide Flask by default, a Python virtual environment was created to manage dependencies in an isolated manner. This ensures that system-level packages are not affected and follows best practices for Python development.

The virtual environment was created and activated, after which Flask was installed and the application was executed:

```
python3 -m venv venv
source venv/bin/activate
```

```
pip install flask
python app.py
```

This setup allowed the Flask-based control panel to run reliably within the virtual machine.

Internally, load is generated using the `stress` tool. Each user interaction results in the creation or termination of stress processes, allowing fine-grained control over CPU utilization. This design enables gradual load variation, making it easier to observe the precise point at which scaling is triggered.

The integration of a web-based interface significantly improves the demonstrability of the system, as it allows real-time interaction and visualization without requiring command-line operations.

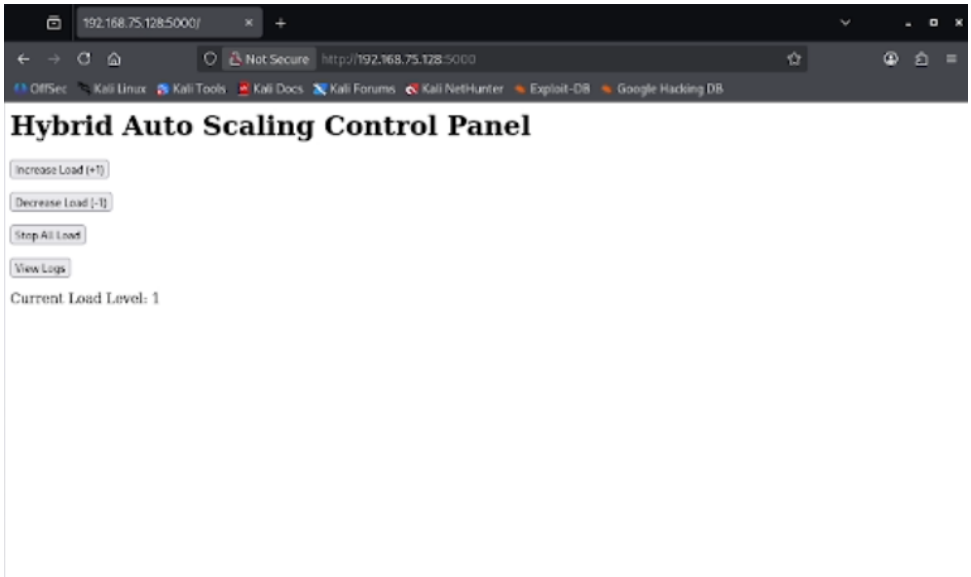


Figure 7: Interactive Web-Based Load Control Panel

7 Sample Application Deployment

A simple web application was deployed on each cloud instance using Apache. The application is intentionally lightweight and serves the purpose of verifying that instances created during scaling are functional and capable of handling requests.

8 Resource Migration and Cloud Bursting

The concept of resource migration in this implementation is realized through a cloud bursting approach. Instead of physically transferring processes or application state from the local VM to the cloud, the system shifts computational responsibility by provisioning additional resources in the cloud when local capacity is exceeded.

This approach is widely adopted in real-world systems because it avoids the complexity associated with stateful migration while still achieving scalability. It allows existing systems to extend seamlessly into the cloud during peak demand, thereby ensuring performance without requiring a complete redesign of the application architecture.

9 Testing and Validation

The system was tested by gradually increasing load through the control panel. As load increased, CPU utilization rose accordingly, eventually crossing the defined threshold. At this point, new instances were automatically provisioned in the cloud. When the load was subsequently reduced, the system observed a decrease in utilization, and after the cooldown period, unnecessary instances were terminated.

It was observed that there is a slight delay between issuing a scaling command and the actual creation of instances, which is expected due to cloud provisioning time. However, this delay was not significant in scale-up operations. In contrast, scale-down operations took comparatively more time, as instances are not terminated immediately. This behavior is consistent with auto-scaling principles studied earlier, where scale-down is intentionally delayed to avoid frequent oscillations and ensure system stability.

This behavior confirms that the system successfully implements both scale-up and scale-down operations in a stable and controlled manner.

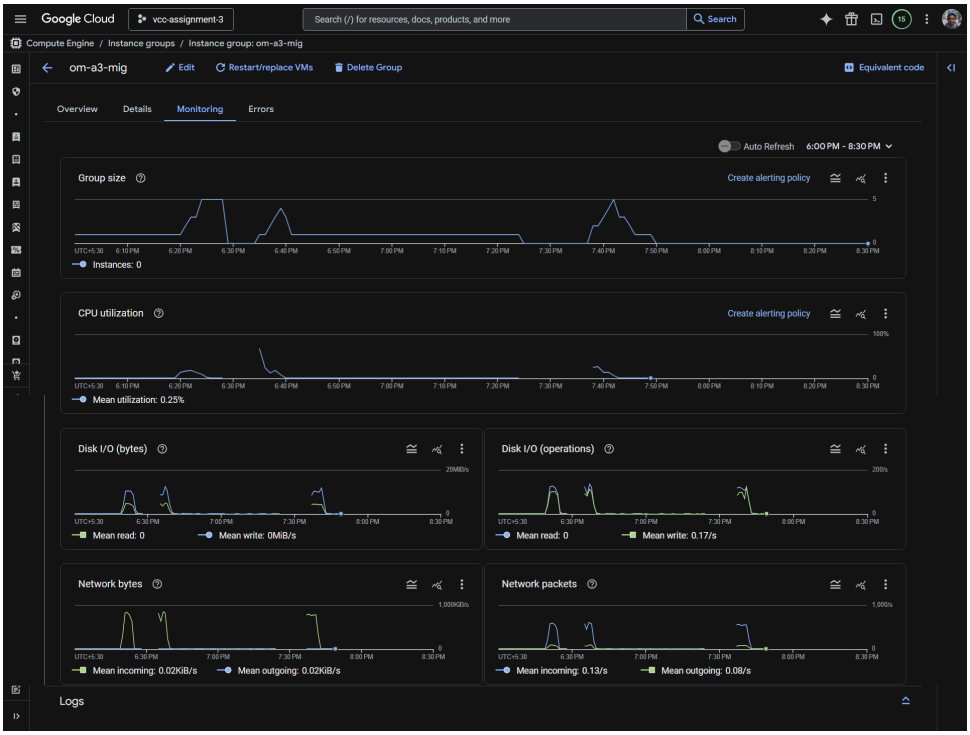


Figure 8: Observed Scaling Behaviour During Testing

10 Architecture Overview

The overall architecture of this system follows a hybrid cloud approach, where a local virtual machine is connected with a public cloud platform to enable dynamic scaling based on resource usage. The system is divided into three main parts: the local execution and monitoring layer, the decision-making layer, and the cloud infrastructure layer. These components work together to detect high load on the local system and extend resources to the cloud when required.

The local setup consists of a host machine running VMware Workstation Pro, inside which a Kali Linux virtual machine is deployed. This VM acts as the main environment where load is generated, monitored, and scaling decisions are taken. The use of virtualization ensures isolation and makes the setup closer to real-world deployment scenarios.

Within the Kali VM, I implemented a load generation mechanism using a Flask-based

control panel along with stress processes. This allows controlled increase and decrease of CPU load instead of applying a sudden heavy load, making it easier to observe the system behavior. Along with this, a custom monitoring script continuously tracks CPU and memory usage using lightweight system commands. This approach avoids unnecessary overhead while still providing accurate real-time metrics.

The decision engine uses these metrics to determine when to scale. If either CPU or memory usage exceeds 75%, a scale-up operation is triggered. On the other hand, if CPU usage drops below 30%, a scale-down operation is performed. This design is slightly asymmetric, as scale-up considers both CPU and memory to capture any bottleneck, while scale-down is based only on CPU to avoid removing instances too early. A cooldown period of 60 seconds is also enforced to prevent frequent scaling actions due to small fluctuations.

Once a decision is made, the system uses the gcloud CLI to communicate with Google Cloud Platform. The command resizes a managed instance group, which is responsible for creating or deleting virtual machines automatically. The use of an instance template ensures that every new VM is configured in the same way, including automatic deployment of an Apache web server using a startup script.

On the cloud side, the managed instance group maintains a minimum and maximum number of instances and dynamically adjusts based on the scaling commands received. During scale-up, new instances are created and become ready to serve requests, while during scale-down, unnecessary instances are removed to save resources.

Overall, this setup demonstrates a cloud bursting model, where the local system handles normal workload and offloads excess load to the cloud when resource usage becomes high. Instead of moving applications, additional resources are created in parallel, which is a practical and commonly used approach in real-world systems. The architecture ensures proper flow from load generation to monitoring, decision-making, and finally cloud-based scaling.

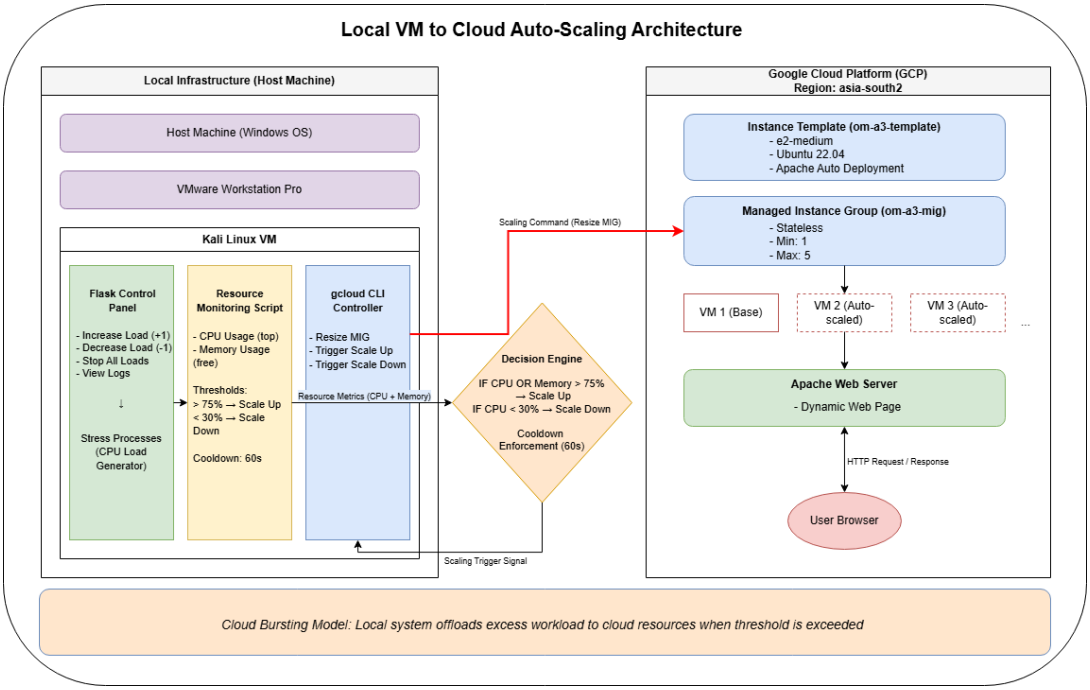


Figure 9: System Architecture

11 Source Code Repository

The source code repository contains all components developed for this assignment, including the resource monitoring script, Flask-based load generation application, cloud configuration details, and supporting documentation. The repository reflects the complete implementation of the hybrid auto-scaling system from a local virtual machine to Google Cloud Platform.

`https://github.com/OmKumar2004/VCC-Assignment-3`

12 Recorded Video Demonstration

A video demonstration has been recorded to showcase the complete workflow of the system. The video includes the setup of the local virtual machine, real-time resource monitoring, controlled load generation using the Flask interface, and the automatic scaling of cloud instances based on defined thresholds. Both scale-up and scale-down behaviors are demonstrated along with explanation of the underlying logic.

`https://drive.google.com/file/d/1L1kJaTqIyAylF_KUiApQ1LNuhWeYFMeY/view?usp=sharing`

13 Conclusion

This assignment demonstrates the implementation of a hybrid cloud system capable of dynamically scaling based on resource utilization. By combining lightweight monitoring, controlled load generation, and automated cloud provisioning, the system effectively illustrates the principles of auto-scaling and cloud bursting.

The design choices made in this implementation prioritize simplicity, clarity, and demonstrability while still maintaining alignment with real-world practices. The inclusion of bidirectional scaling and stabilization mechanisms further enhances the robustness of the system.

14 Plagiarism Declaration

I hereby declare that this assignment is entirely my own work. I have not copied from other students. I referred to online resources only for understanding concepts and implementation steps. Any instance of plagiarism may result in the submission being declared void.